

Grado en Ingeniería Software

Tema 3

Decisiones de Diseño y Conocimiento Arquitectónico

Índice de la Presentación

- Introducción
- Caracterización de Decisiones
- Vista de la decisión y Procesos
- Dependencias y traceabilidad
- Herramientas de AKM
- Reflection
- Conclusiones

Introducción

- Una **Arquitectura Software** debe ser considerada como el resultado de un conjunto de decisiones de diseño (ADD – Architectural Design Decisions) más que como un conjunto de componentes y conectores (Bosch, 2004).
- Las decisiones de diseño permiten conocer los motivos subyacentes de una decisión tomada y las alternativas evaluadas.
- Permiten seguir mejor la evolución de una arquitectura software.

Introducción

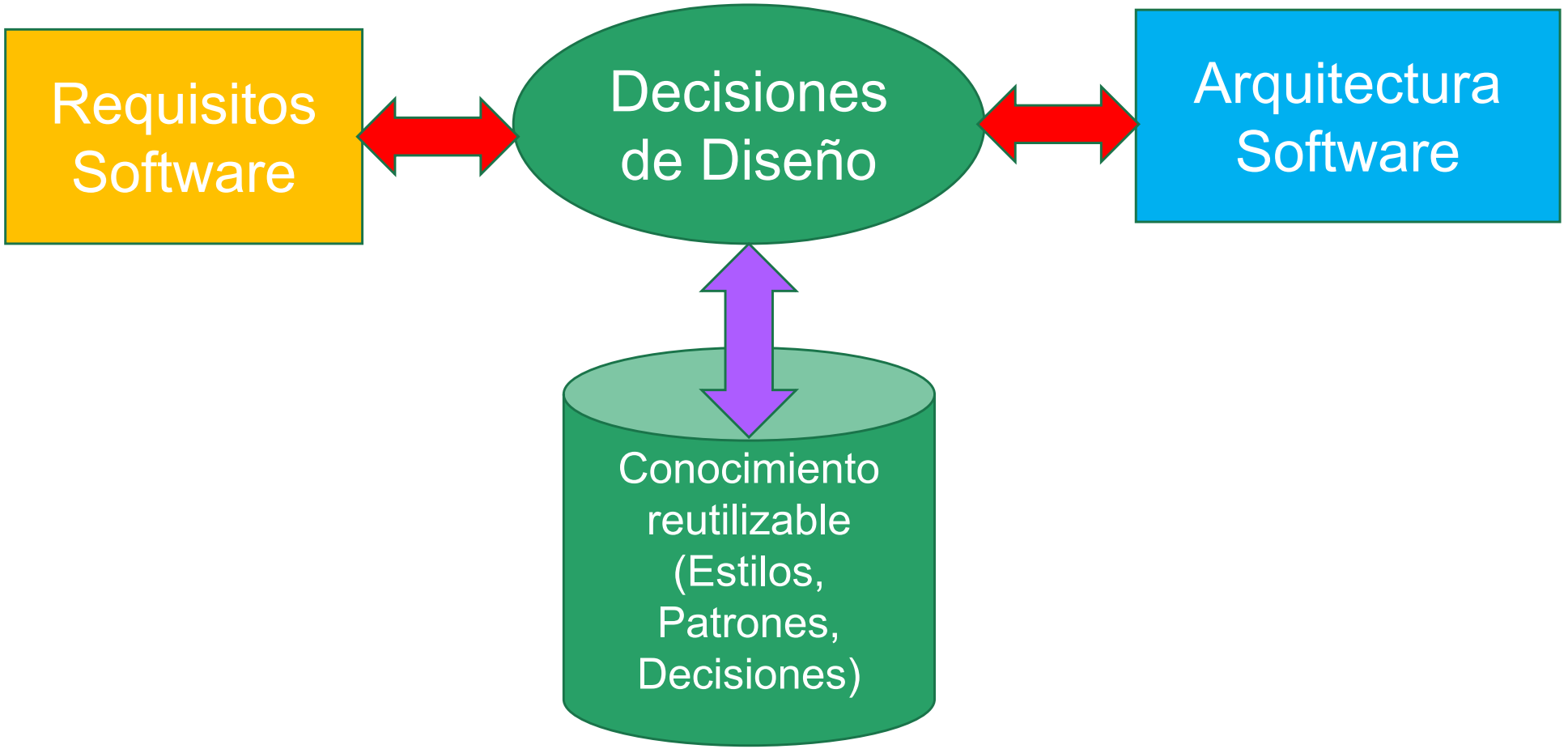
- Son necesarias para comprender las alternativas evaluadas durante el proceso de diseño.
- Son un conocimiento implícito, subyacente y tácito que reside en la mente de los arquitectos software y que es necesario hacer explícito para que pueda ser (re)utilizable y transferible.
- Es necesario cambiar los hábitos de diseño actuales para favorecer su captura.

Introducción

- Las ADD cubren el hueco entre requisitos y diseño
- Facilitan que el conocimiento experto no se evapore
- Son útiles para enseñar a diseñadores poco expertos
- Permiten comprender el motivo de decisiones buenas y malas
- Son reproducibles

Introducción

Las decisiones de diseño cubren el hueco entre los requisitos software (de donde provienen) y las arquitecturas software (el resultado de tomar un conjunto de decisiones). Las decisiones capturadas se pueden considerar un conocimiento reutilizable



Estándar ISO/IEC/IEEE 42010:2022

- Práctica recomendada para la descripción de sistemas intensivos software
- Las decisiones de diseño son elementos de primera clase definidos en el estándar ISO/IEC/IEEE 42010:2022
- Se capturan y documentan junto con las arquitecturas software
- Mantiene el modelo de vistas arquitectónicas

Caracterización de Decisiones

- Caracterizar el conocimiento que se pretende almacenar sobre las decisiones de diseño es el primer paso.
- Tres aproximaciones:
  - Plantillas de texto y ficheros MD
  - Ontologías
  - Anotaciones en código

Para capturar el conocimiento de las ADD es necesario caracterizarlas o describirlas de alguna manera ya que se expresan en lenguaje natural

Caracterización de Decisiones

- Las plantillas de texto definen un conjunto de atributos que capturan el conocimiento de las decisiones y su motivación.
- Algunas aproximaciones utilizan un conjunto menor de atributos y otros opcionales definidos por el arquitecto software para reducir el coste de capturar un gran número de atributos.
- Las plantillas deben ser flexibles en cuanto a número de ítems o atributos a capturar

Caracterización de Decisiones

- Atributos para describir una decisión
  - Nombre: Asignar un nombre corto a la decisión
  - Descripción de la decisión: Descripción corta pero explicativa
  - Estatus: Si la decisión esta "*pendiente/aprobada/rechazada*"
  - Categoría
  - Lógica: El mótivo de tomar dicha decisión
  - Responsable
  - Fecha
  - Pros/Cons
  - Etc…

Si una decisión de diseño fuera seleccionar el estilo arquitectónico Cliente/Servidor podemos caracterizar esta con la lista de ítems o atributos que se muestran en la transparencia.

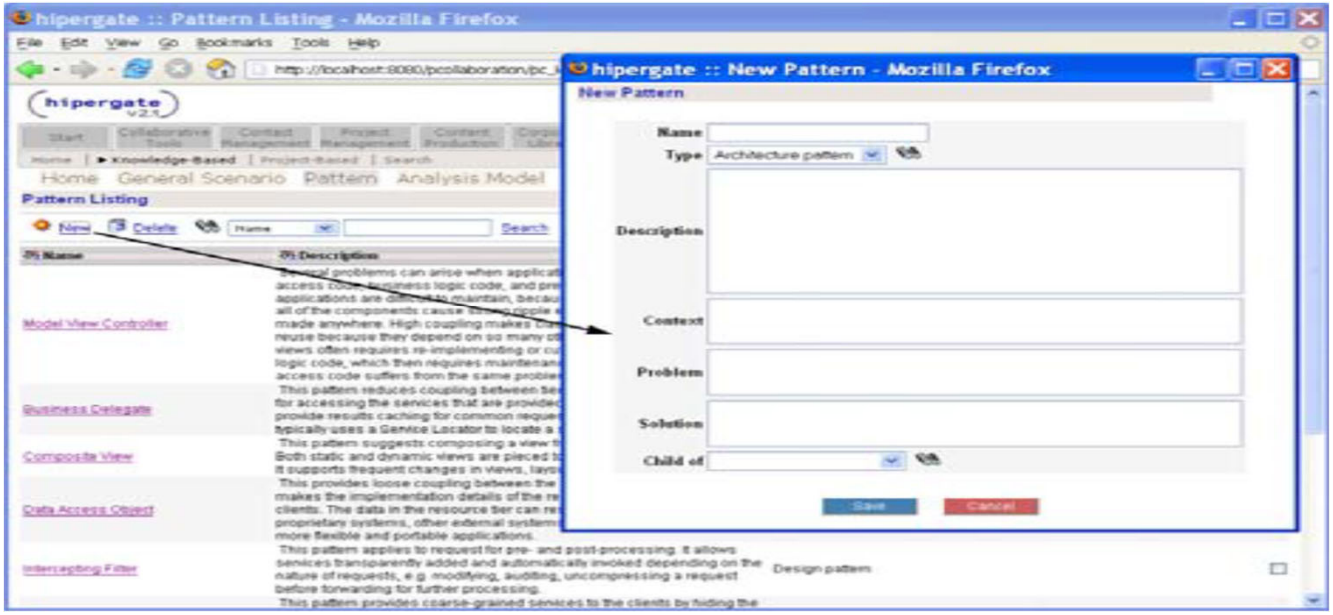
Caracterización de Decisiones (Tyree y Akerman, 2005)

14 atributos fijos para caracterizar las DDs

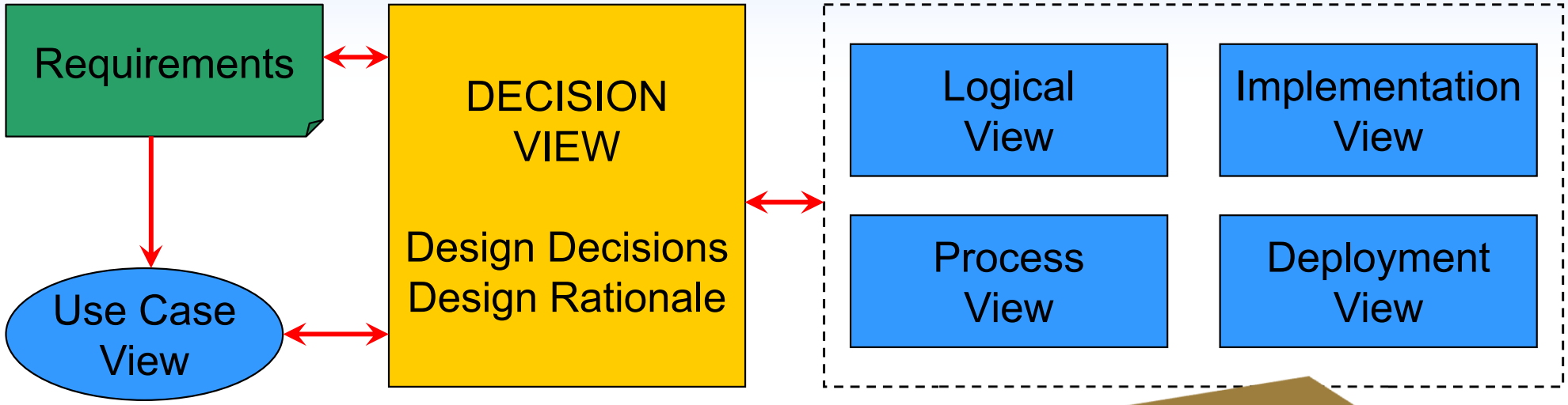
- Issue: Problema de diseño
- Decision: Descripción de la decisión
- Status: Pendiente, Decidida, Aprobada
- Constraints: Restricciones respecto al entorno
- Positions: Alternativas
- Related decisions
- Related requirements
- Related artifacts: otros elementos del diseño

Caracterización de Decisiones
(Captura de AK con la herramienta PAKME, 2006)

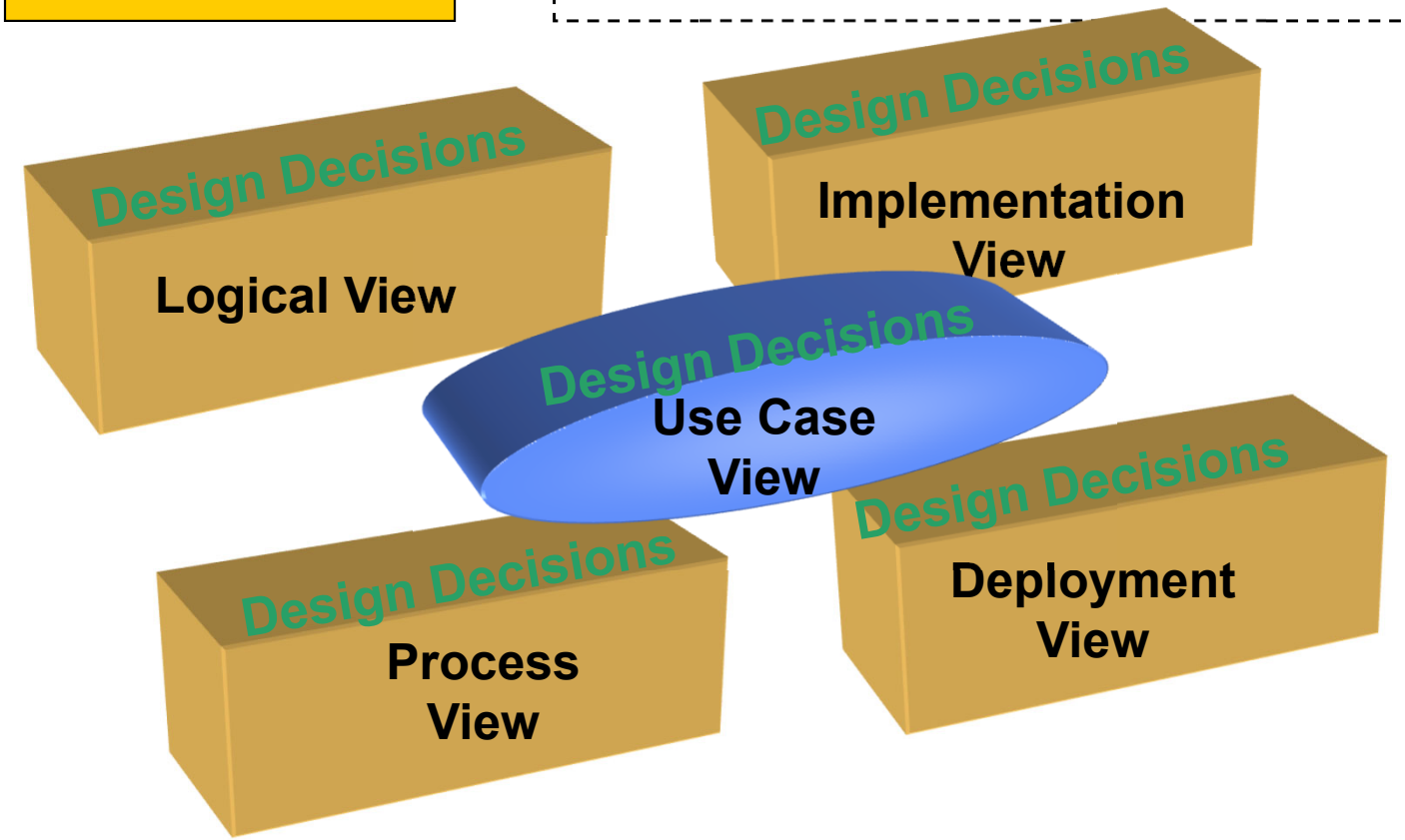
- Basada en la plantilla de Tyree
- Relación con atributos de calidad (QA)
- Soporta compartición de conocimiento (knowledge sharing)
- Captura patrones de diseño (design patterns)
- Permite describir “scenarios”
- PAKME: Captura, Mantenimiento, Recuperación, Presentación



Vista de la Decisión



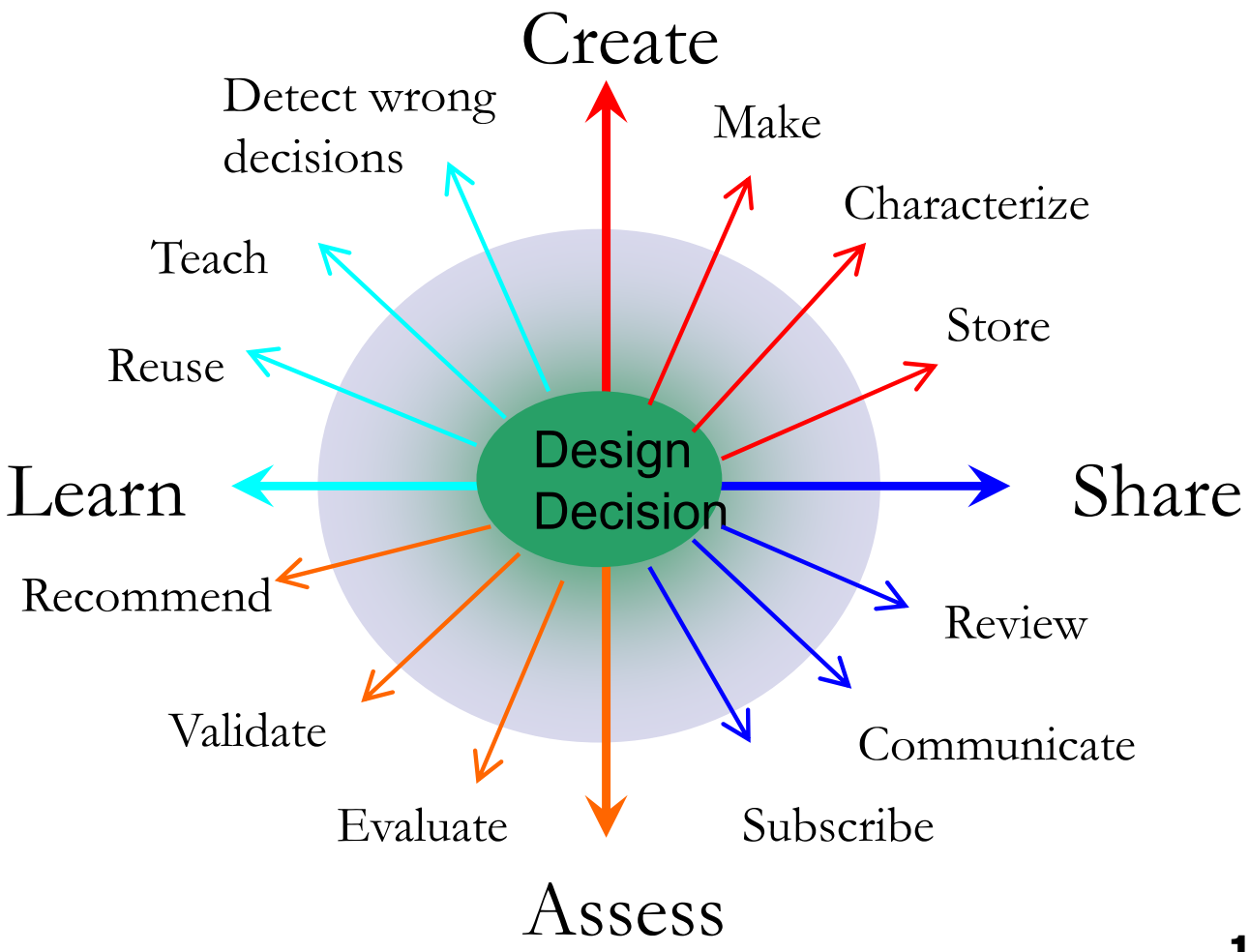
La vista de la decisión es una nueva vista arquitectónica que es transversal a las restantes vistas del modelo 4+1



Vista de la decisión y Procesos

- La captura y representación de ADDs es solamente una de las actividades de la Gestión del Conocimiento Arquitectónico (**AKM – Architecture Knowledge Management**)

Son los procesos que surgen para crear una decisión, compartirla, utilizarlas para asesorar a arquitectos menos expertos y aprender de ellas.



Vista de la decisión y Procesos

- Capturar las ADD y documentarlas junto con su arquitectura es el primer paso.
- Establecer relaciones de traceabilidad con requisitos y arquitecturas.
- Compartir las decisiones con otros stakeholders (“**knowledge sharing**”)
- Mantener y evolucionar las decisiones a lo largo del tiempo (“**Decision History**”) en un histórico de decisiones

Dependencias y Evolución

- Las herramientas de AKM suelen incluir forma de establecer dependencias explícitas a requisitos y arquitecturas desde las decisiones.
- Las dependencias entre decisiones, requisitos y arquitecturas permitem establecer una traceabilidad completa para:
  - Averiguar las causas y orígenes de cambios en el diseño ("**backward traceability**").
  - Estimar el análisis de impacto en el diseño por cambios en los requisitos ("**forward traceability**").

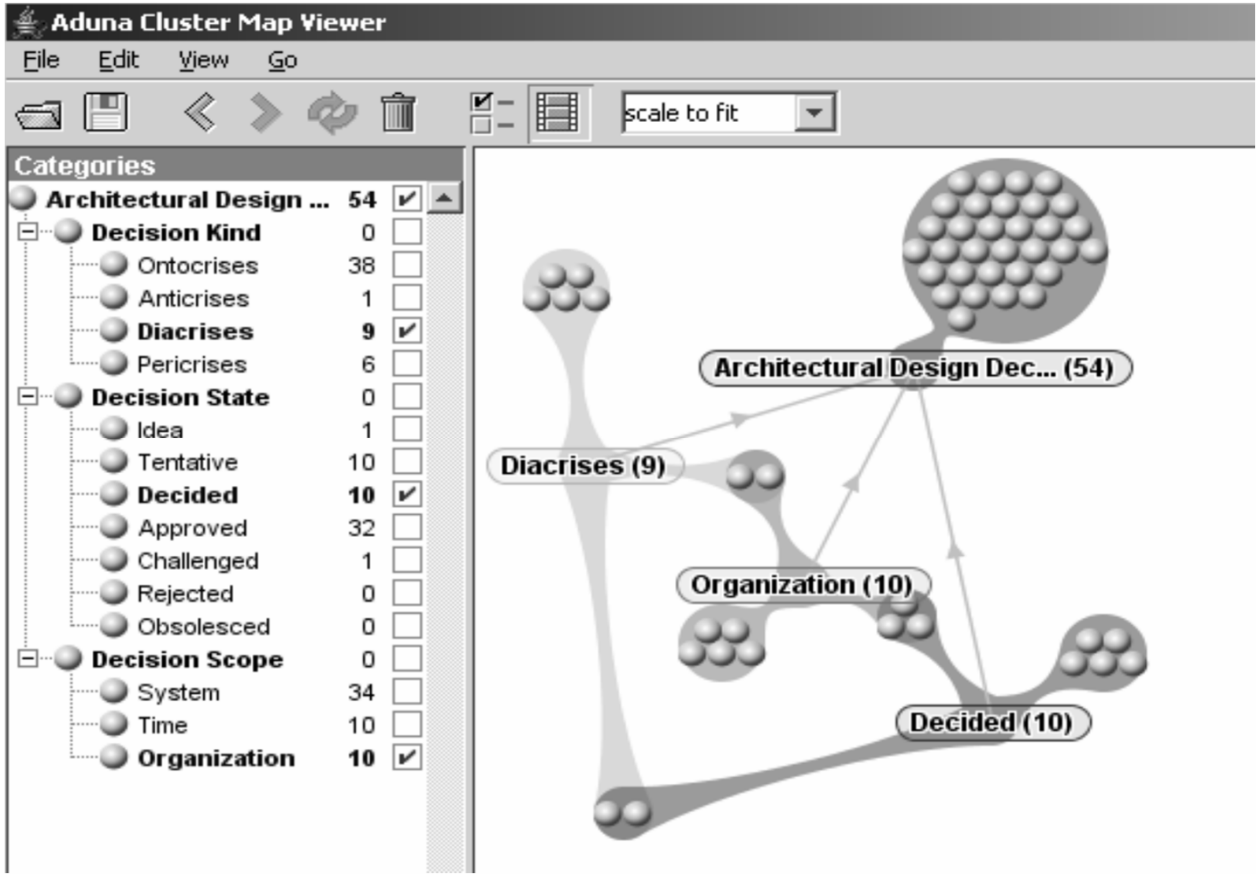
ADD: Traceabilidad Interna

- Las dependencias entre decisiones de diseño constituyen una traceabilidad interna para conocer el impacto en una decisión cuando otra se modifica.
- Si la granularidad de las decisiones es fina, la red de dependencias entre decisiones puede crecer enormemente.
- Limitar decisiones a nivel a clase en la arquitectura.

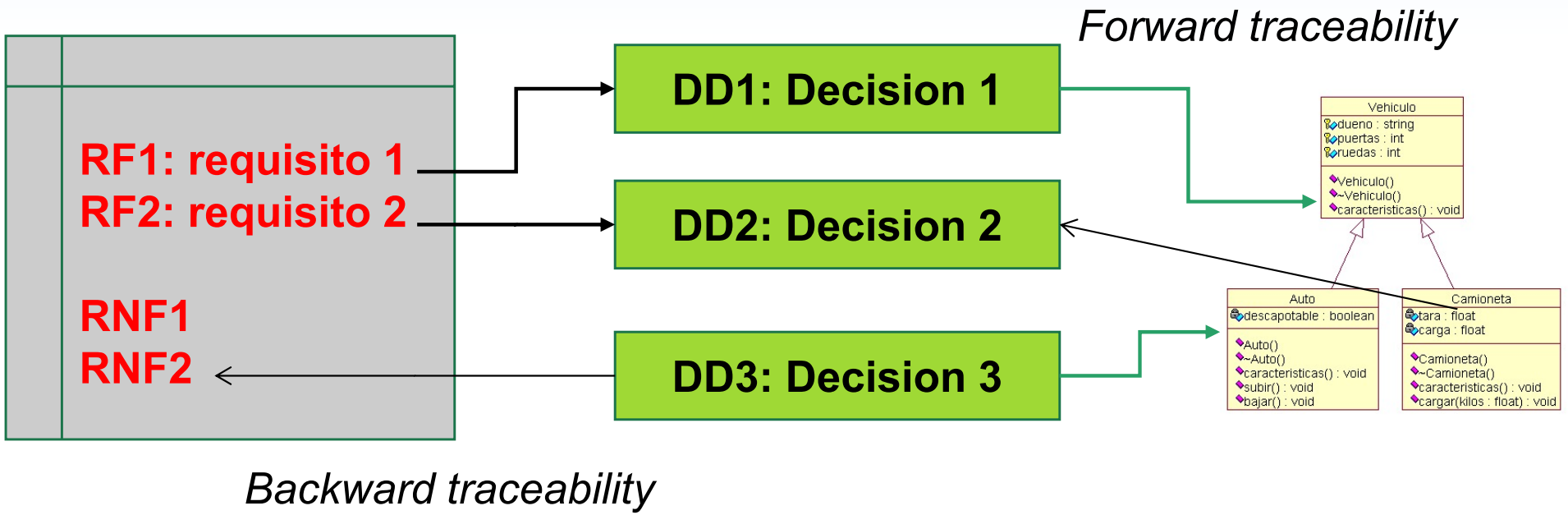
Traceabilidad Interna (Ontologías)

Decision A relacionada con **Decision B**
Decision A excluye a **Decision B**
Decision A permite **Decision B**
Decision A es conflicto con **Decision B**

Pueden existir conflictos entre decisiones incompatibles o contrarias que tenemos que resolver. O bien, una decisión puede excluir a otra (e.g. Si utilizamos una arquitectura de 2 capas no es posible incluir el middleware X)



ADD: Traceabilidad Externa



Gracias a que las decisiones se sitúan entre los requisitos y las arquitecturas podemos definir enlaces de traceabilidad para: BACKWARD: conocer el origen (el requisito) de una decisión de diseño mal tomada (un patrón mal seleccionado) o bien conocer FORWARD: el impacto en el diseño de un cambio en los requisitos

Herramientas de AKM

- Desde 2005 se han creado un conjunto de herramientas de investigación prototipo para la captura y compartición de decisiones de diseño.
- **PAKME**: Herramienta Web para la captura de decisiones basada en la plantilla de Tyree & Akerman (2005) con soporte para atributos de calidad (Babar et al).
- **Archium**: Herramienta de modelado que permite la captura de DD con avisos sobre decisiones incompatibles.

Herramientas de AKM

- **AREL/eAREL**: Herramienta de modelado UML que permite incluir DD en los modelos de clases. Soporta dependencias e histórico de decisiones.
- **ADDSS**: Herramienta Web para la captura de conocimiento. Soporta personalización de atributos, evolución de las arquitecturas y dependencias. Las últimas versiones incorporan mecanismos de compartición.
- **The Knowledge Architect**: Herramienta plug-in Word que permite marcar las decisiones de diseño con otros elementos del documento Word.

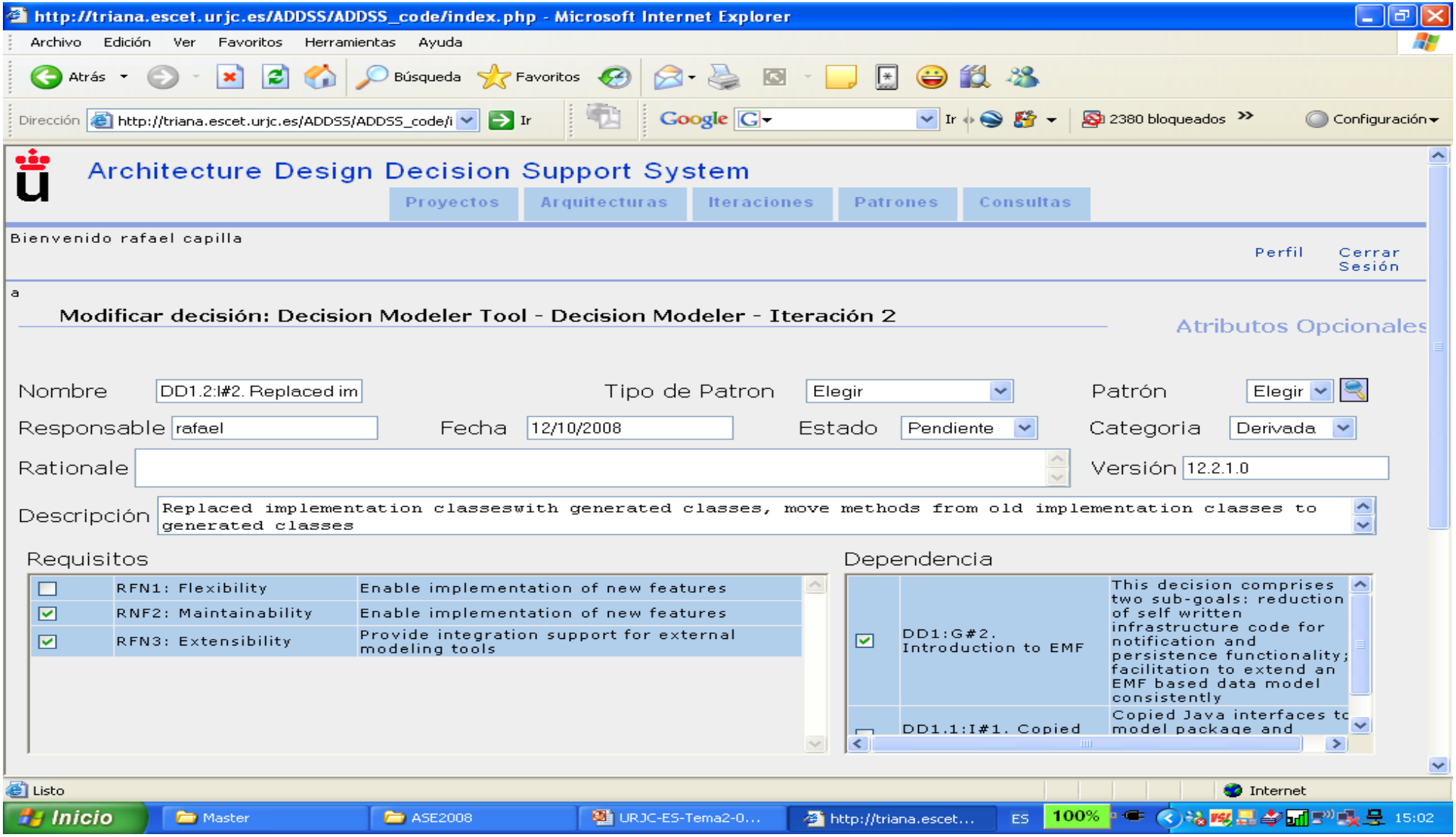
Herramientas de AKM

- **EAGLE**: Herramienta Web para compartir conocimiento. Incluye RSS y gestión de usuarios. Basada en la plataforma Hipergate.
- **AdWiki**: Herramienta Web similar a EAGLE. Almacena más de 400 decisiones SOA.

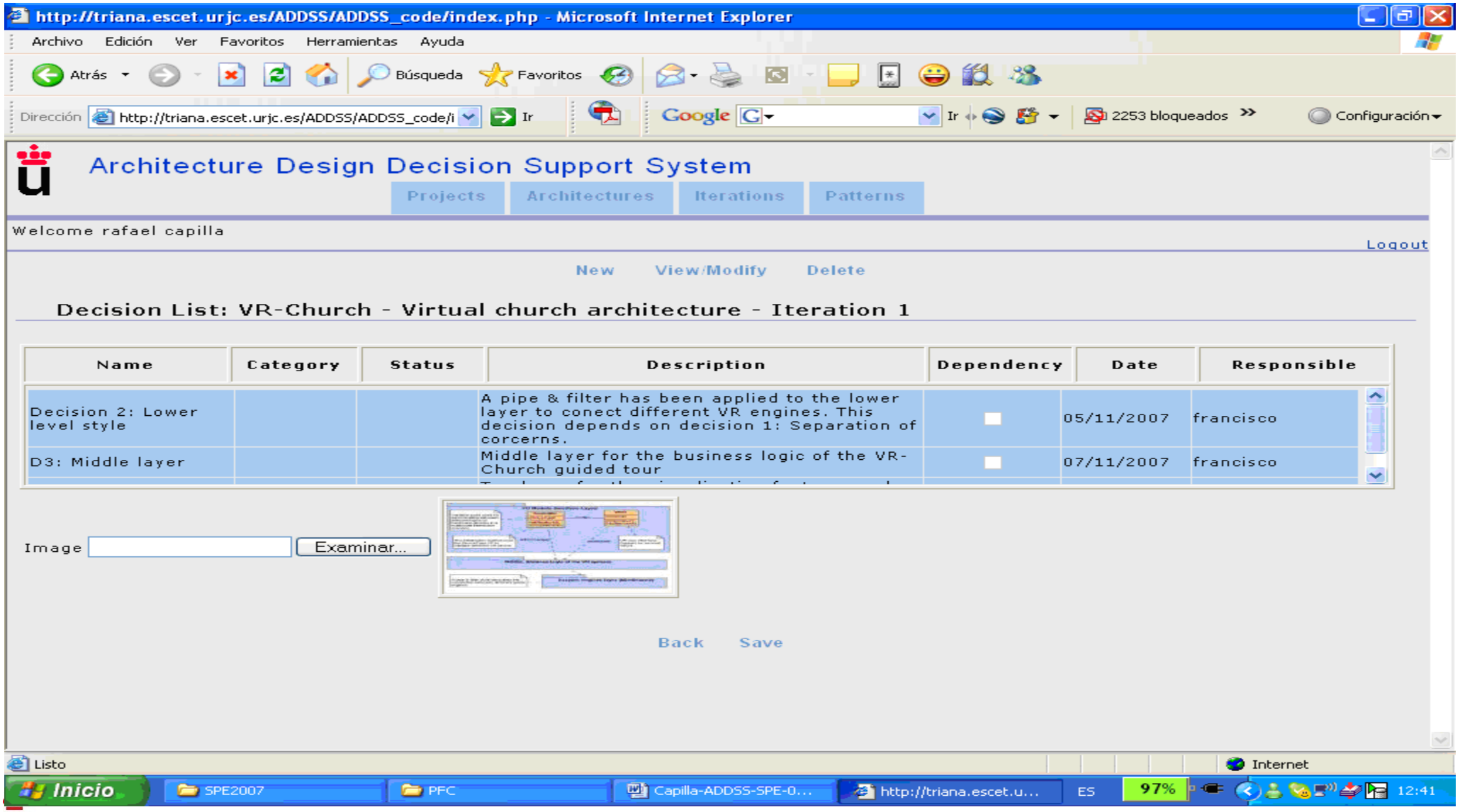
Herramienta ADDSS (2006-2011)

- Architecture Design Decision Support System
  - Captura decisiones con sus arquitecturas.
  - Soporte para vistas arquitectónicas.
  - No permite modelado UML.
  - Permite atributos opcionales y obligatorios para describir las decisiones.
  - Categoría y estatus de las decisiones.
  - Dependencias a otras decisiones.
  - Dependencias entre requisitos, decisiones y arquitecturas.
  - Soporte RSS y Chat.
  - Documentación PDF automática.
  - Visualiza la evolución de las arquitecturas.

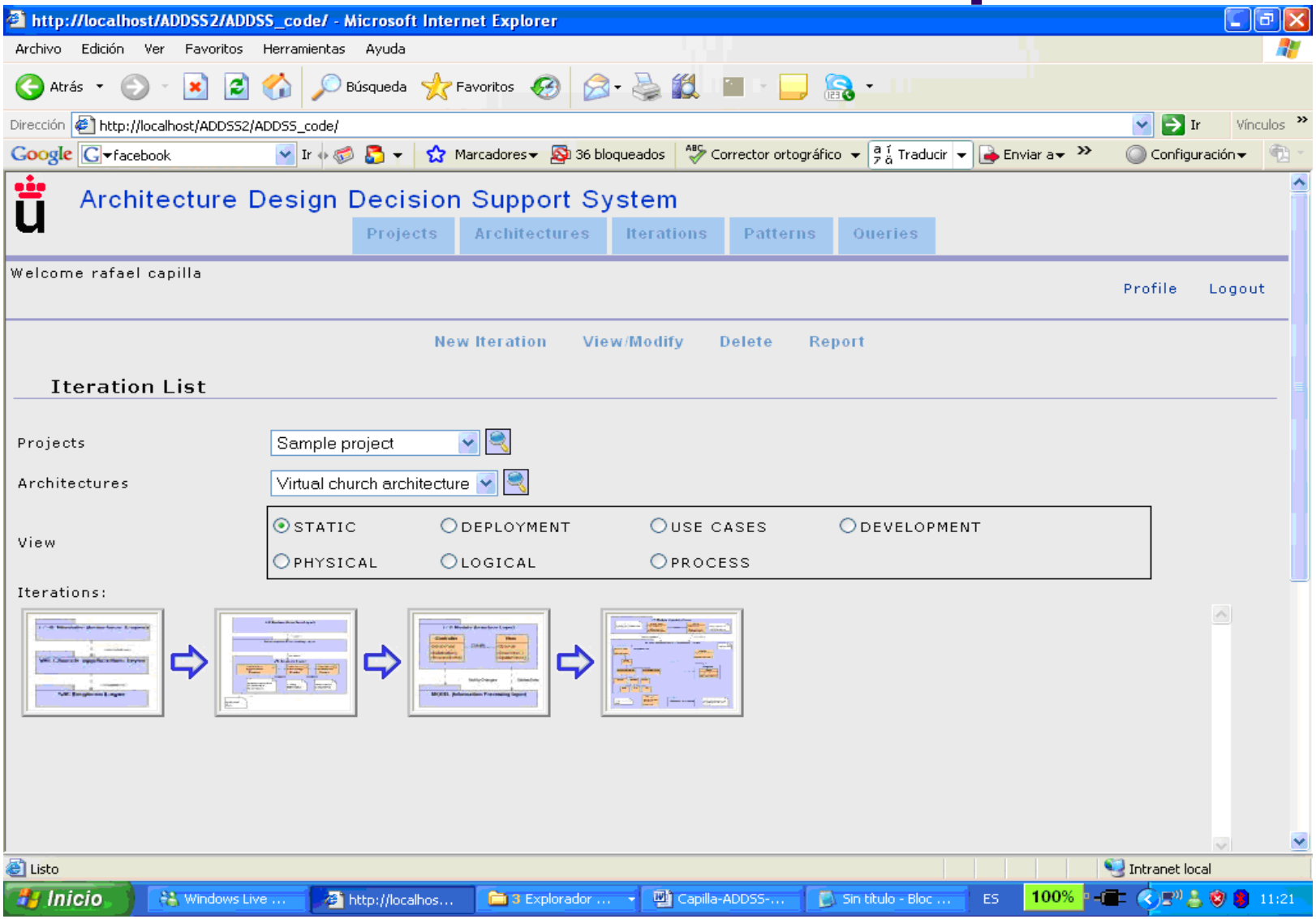
ADDSS: Decisiones y traceabilidad



ADDSS: Traza a Arquitecturas



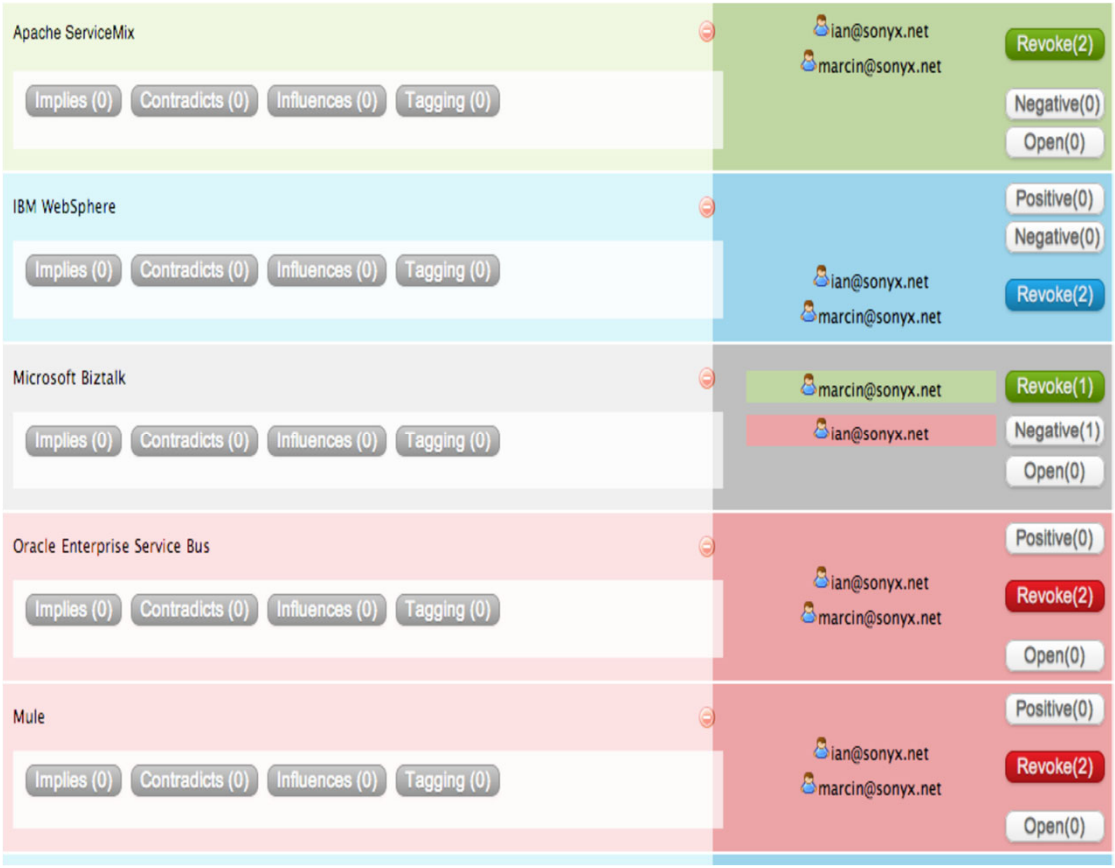
ADDSS: Evolución de la Arquitectura



Herramienta SAW Software Architecture Warehouse (Nowak, Pautasso)

- Captura, gestiona, comparte y analiza AK en formato Wiki
- Búsqueda por texto y navegación por tags
- Gestiona consenso entre decisiones
- Capacidades colaborativas y modelo fuzzy de decision



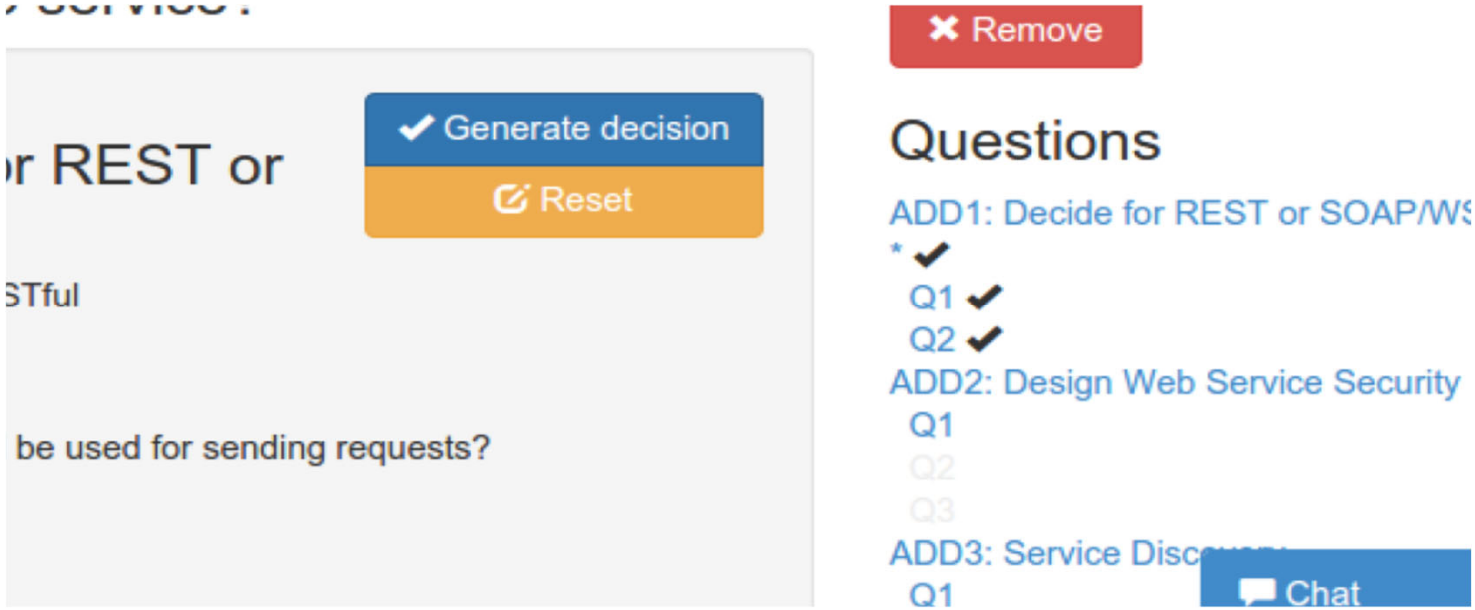


Herramientas de AKM

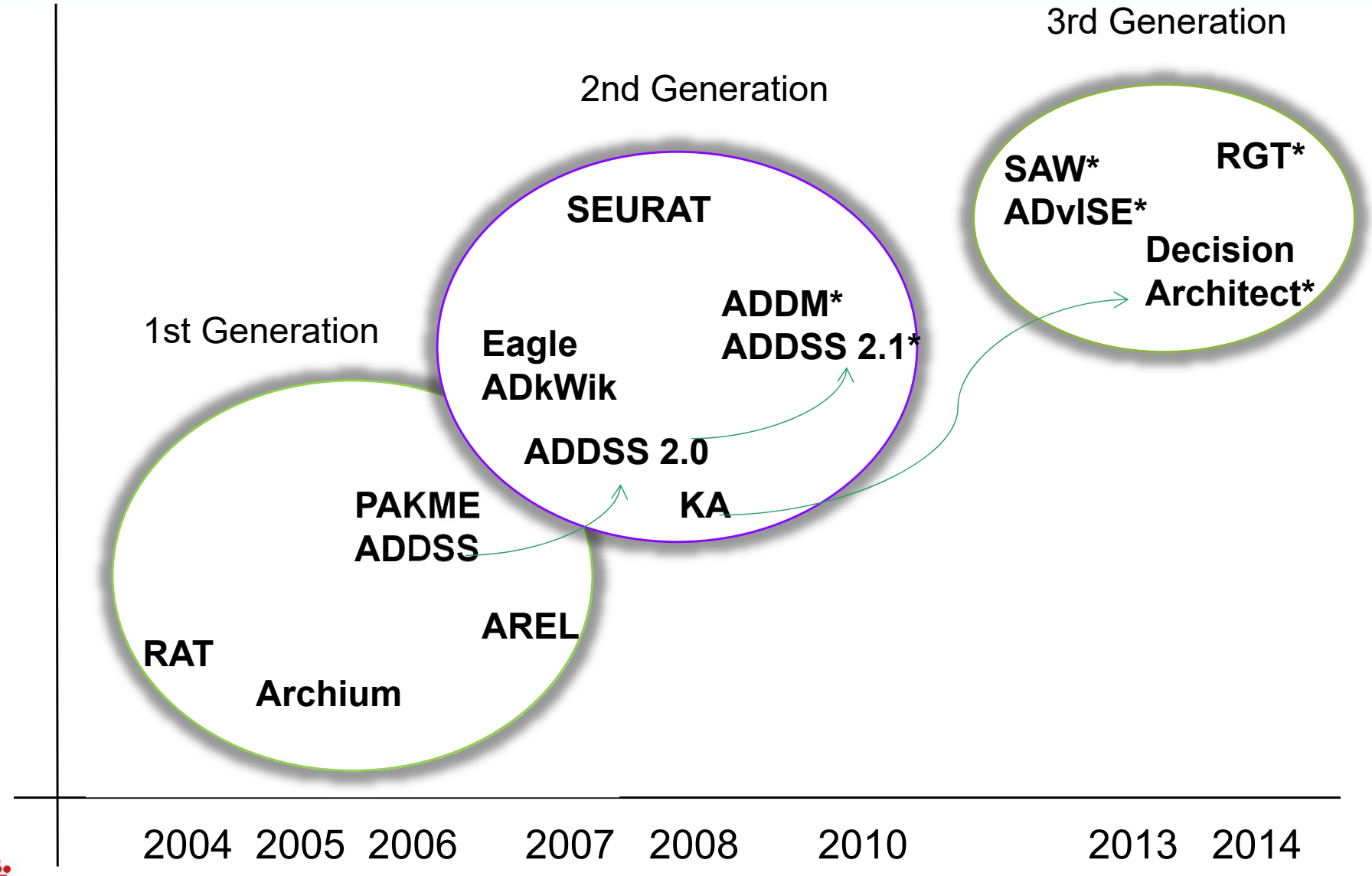
- **ADvISE (Architectural Design Decision Support Framework):** Soporta la captura y representación de ADD mediante el método QOC ("Question-Option-Criteria"). Permite crear decisiones reutlizables con un cierto grado de incertidumbre y generar preguntas basadas en las decisiones tomadas.
- Junto con la herramienta VbMF (View-based Modeling Framework) proporciona una vista de componentes y conectores (C&C) de la arquitectura y enlaces entre arquitecturas y decisiones.

Herramientas de AKM

- **CoCoADvISE (Constrainable Collaborative Architectural Design Decision Support Framework):** Proporciona un soporte semi-automático para toma de decisiones y documentación.
- Permite colaboración entre usuarios.
- Utiliza modelos de ADD reutilizables mediante QOC.
- Extiende QOC con dependencias entre opciones y decisiones.

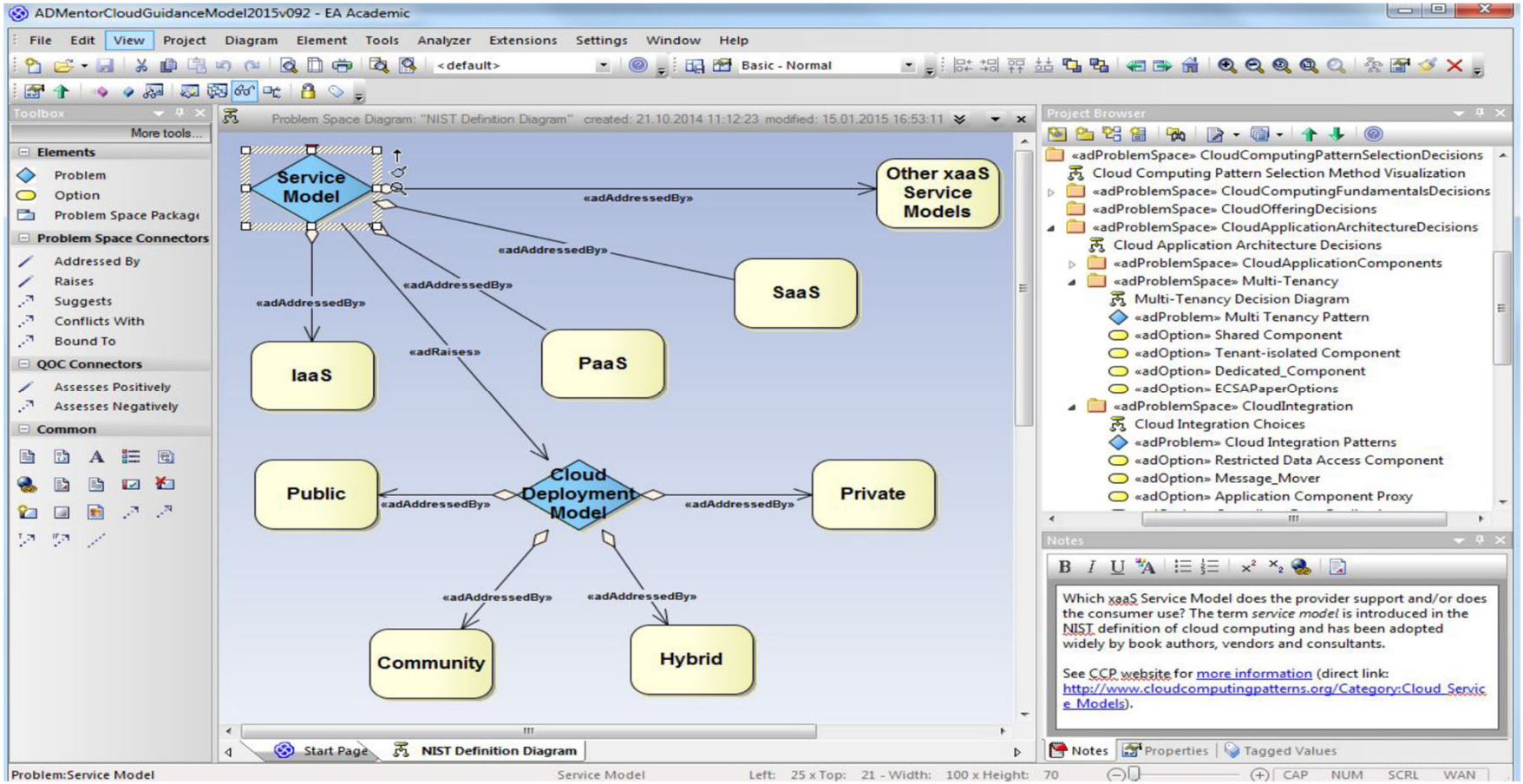


Generaciones de Herramientas de AKM



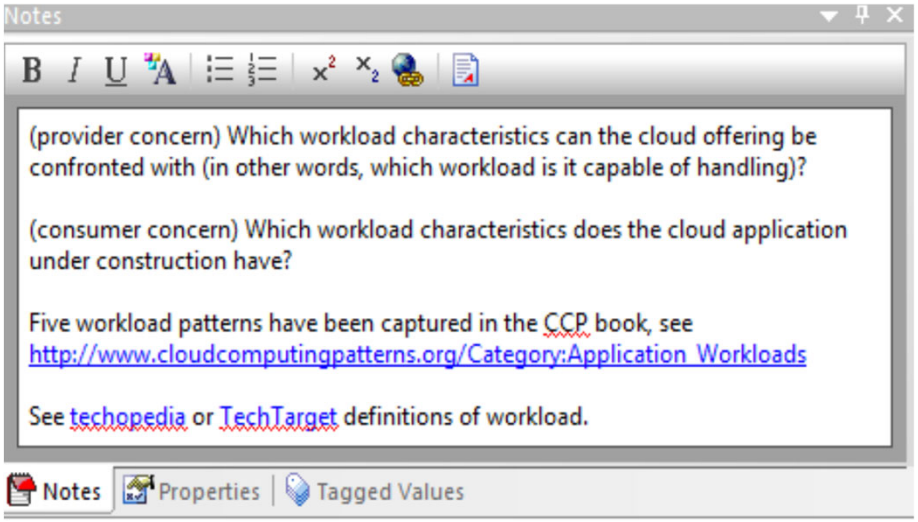
Herramienta ADMentor

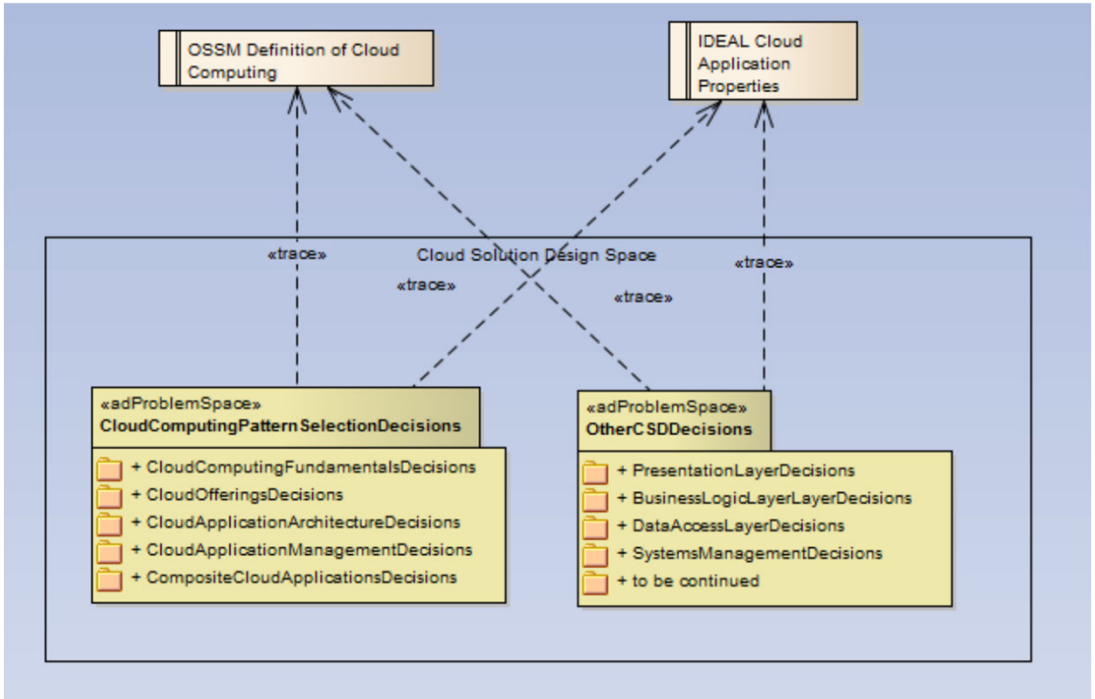
- Se ejecuta sobre Enterprise Architect (EA)
- Enlace a patrones de diseño
- Proporciona analíticas



Herramienta ADMentor

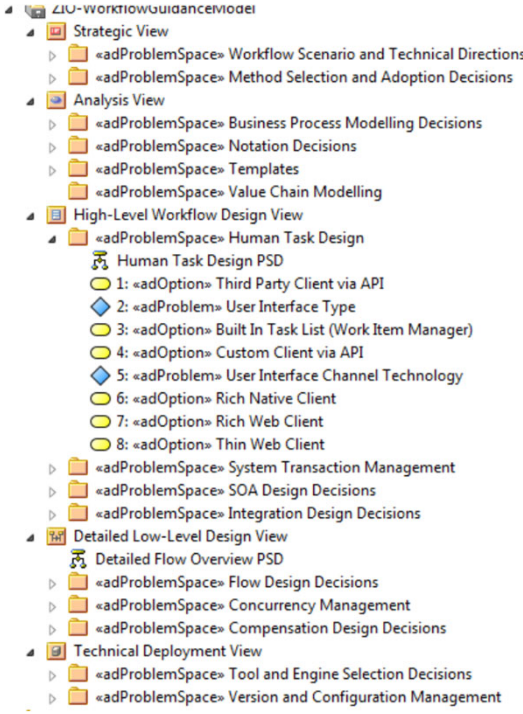
- Texto enriquecido y enlaces Web
- Modelado de AK aplicado en la arquitectura de la empresa ABB

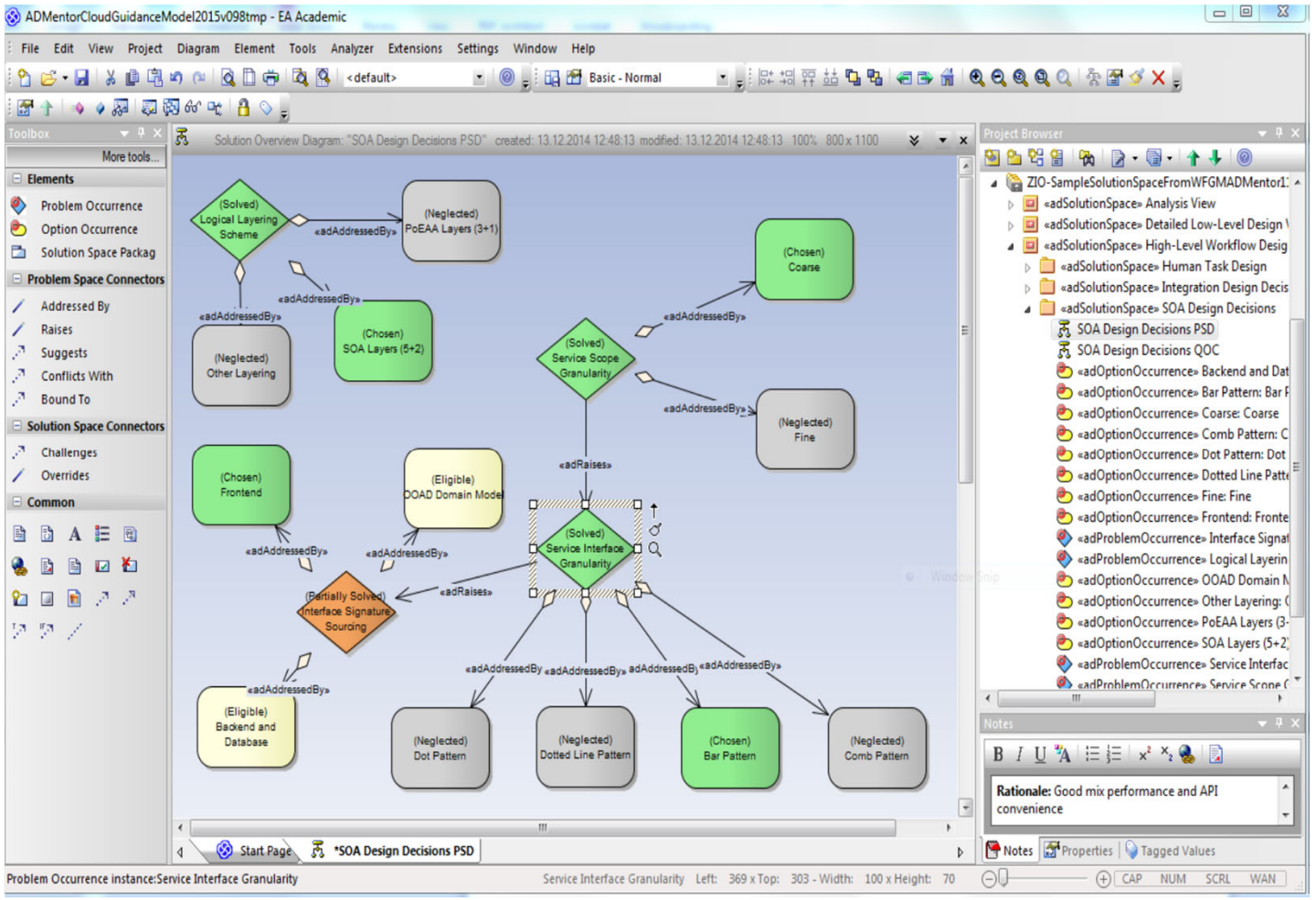




Herramienta ADMentor

- Método QOC (Question-Option-Criteria) para describir el espacio del problema
- Interfaz con el espacio de la solución (decisiones tomadas)





Formato de captura MADR

- **MADR: Markdown Architectural Decision Records**
- Es un formato libre
- Disponible en: https://adr.github.io/madr/
- Proporciona ficheros .MD en texto plano para capturar la información de las decisiones
- Permite enlazar decisiones por su ID
- No todos los ítems de captura son obligatorios
- Conexión con GitHub

Formato de captura MADR

{short title, representative of solved problem and found solution}

Context and Problem Statement: Describe the context and problem statement

Decision Drivers: son los requisitos que motivan una decisión

- {decision driver 1, 2, 3…}

Considered Options: Alternativas de diseño

- {title of option 1, 2, 3}

Formato de captura MADR

Decision Outcome: Chosen option: "{title of option 1}", because {justification. e.g., only option, which meets k.o. criterion decision driver | which resolves force {force} | }.

Pros and Cons of the Options

- **{Alternativa de diseño 1}**
- Good, because {argument a}
- Neutral, because {argument c}
- Bad, because {argument d}

Captura de decisiones con MADR

- **Caso de estudio:** Una compañía de productos alimenticios pretende migrar la arquitectura de un sistema monolítico a una basada en microservicios de manera.
  - La arquitectura existente cuenta con una parte de clientes PC y móvil que acceden a los datos de la empresa alojados en 2 BBDD SQL (Clientes, Pedidos).
  - La BBDD de Clientes contiene datos de clientes y pagos mientras que la BBDD de Pedidos se encarga de almacenar los datos restantes.
  - El acceso actual se pretende que sea sustituido por protocolos HTTP/REST mediante un componente Gateway adecuado.
  - La arquitectura existente cuenta con 6 módulos (críticos, semi críticos y no críticos) que son: Clientes, Pedidos, Reparto y Rutas, Estadísticas, Incidencias y Pagos
  - La nueva arquitectura debe contar con los elementos software adecuados para ejecutar los microservicios

Captura de decisiones con MADR

- **Identificar el principal problema de diseño**: Cambio de estilo de la arquitectura de monolítica a una de micro servicios (MS)
- Hay que documentarse sobre el estilo de MS y sus características principales (e.g. modularidad, distribuido, desacoplamiento de clases/funcionalidad, uso de protocolos de Internet).
  - Buscar estilos similares que sea similares o compartan aspectos comunes (Ej. C/S)
  - Sugerir estilos que no tienen nada que ver con el problema de diseño o con otros propuestos lleva a una mala decisión

Captura de decisiones con MADR

- **Captura con MADR (ítems)**
- **Selección-Estilo-Arquitectónico**
  - Status: accepted
  - Date: 2023-10-20
- **Context and Problem Statement:** En primer lugar debemos decidir cual es el estilo arquitectónico más óptimo para realizar el diseño del problema dado.
- **Decision Drivers**
  - RF01: Cambio de arquitectura.
- **Considered Options**
  - 0001-1-Estilo-Por-Capas
  - 0001-2-Estilo-REST
  - 0001-3-Estilo-Por-Eventos



Captura de decisiones con MADR

- **Captura con MADR (ítems)**

Rationale

- **Decision Outcome**

Chosen option: "0001-1-Estilo-Por-Capas", **because** Permite separar los distintos componentes en capas de una forma óptima y nos permite implementar el resto de los problemas aplicando patrones de diseño.



- **Negative Consequences:** Al cambiar una capa afecta a otras



Captura de decisiones con MADR

- **Captura con MADR (ítems)**
- **Pros and Cons of the Options**
  - **0001-1-Estilo-Por-Capas:** Arquitectura la cual separa los componentes de la aplicación en 4 capas, siendo estas la capa de presentación, la capa de lógica de negocio, capa de datos y capa de servicio.
    - Good, because Es escalable.
    - Good, because Permite la utilización de patrones de diseño dentro de cada capa.
    - Good, because Permite tener una estructura organizada.
    - Bad, because Una estructura compleja con muchas capas puede dificultar la comprensión y el mantenimiento.
    - Bad, because La comunicación entre capas puede ser compleja.

Esta opción no tiene sentido

Captura de decisiones con MADR

- **Captura con MADR (ítems)**
- **Pros and Cons of the Options**
- **0001-2-Estilo-REST:** Arquitectura basada en que el cliente envíe peticiones al servidor y este le responda con el resultado de la consulta.
  - Good, because El acceso en el sistema pasará a ser por protocolos HTTP/REST por lo que sería conveniente utilizar este estilo.
  - Bad, because El estilo por capas es mejor ya que permite desacoplar unos módulos de otros.

Descripción poco precisa

Captura de decisiones con MADR

- **Captura con MADR (ítems)**
- **0001-3-Estilo-Por-Eventos:** Arquitectura que permite detectar los eventos, en este caso alguna acción de consulta o de pago, y actuar en consecuencia.
  - Good, because Es una arquitectura muy eficiente y escalable.
  - Good, because Proporciona respuestas a eventos a tiempo real.
  - Bad, because Es compleja de implementar.
  - Bad, because Requiere un diseño cuidadoso de los eventos para garantizar la entrega y el procesamiento de los eventos.

Esta opción no tiene sentido

Consejos para capturar decisiones con MADR

- Los requisitos funcionales y de dominio solo pueden utilizarse **1 vez** (MADR: decisión drivers)
- Describir alternativas de diseño que tengan relación entre ellas y con suficiente detalle (MADR: Considered Options).
  - 0008-1-Chain of Responsibility
  - 0008-2-State Pattern

No están detalladas

Consejos para capturar decisiones con MADR

- Las consecuencias positivas y negativas deben ser de cada alternativa e indicar porque son buenas y malas en el contexto del problema (MADR: Positive/Negative Consequences)

0008-1 – Chain of Responsibility.

Positive consequences

- Good, because it enforces sequential validation and prevents skipping steps.
- Good, because each phase is encapsulated, allowing independent testing and easier evolution of individual steps.

Negative consequences

- Bad, because debugging failures may require inspecting multiple handlers.
- Bad, because the processing order is implicit, so understanding or modifying the sequence may require reviewing the entire chain.

Son genéricas. No están especificadas al contexto del problema

Reflection

- Acerca del razonamiento lógico en la toma de decisiones
- El pensamiento reflectivo (crítico) desafía la toma de decisiones
- La reflexión incrementa la calidad de las decisiones tomadas
  - ¿Porqué hemos tomado una decisión?
  - ¿Es una decisión, buena, mala, adecuada?
  - ¿En qué nos hemos basado para tomarla?
  - ¿Cuál es el contexto del problema que puede afectar a nuestra decisión?

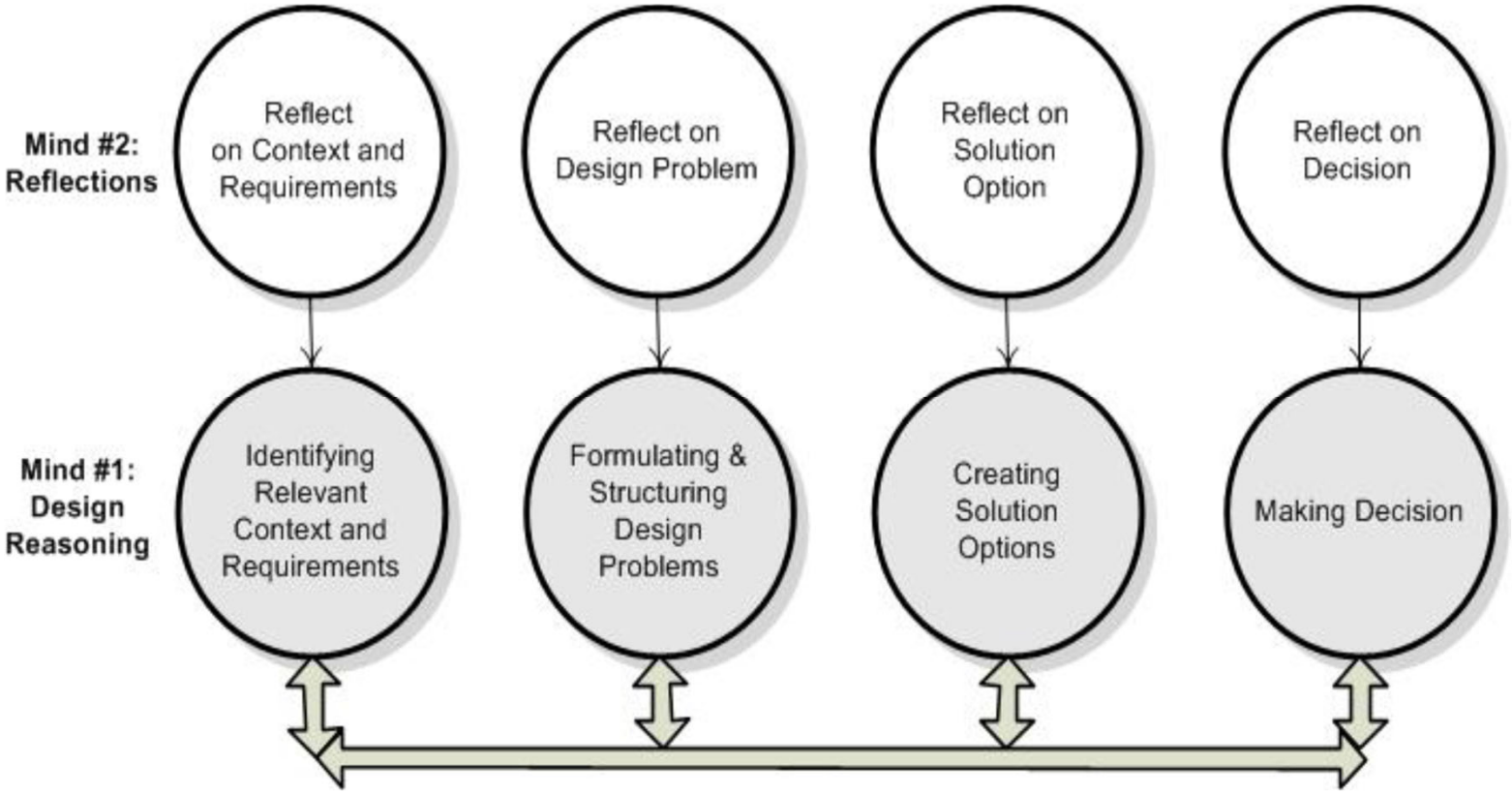
Reflection

- Debemos evitar toma de decisiones sesgadas (ej: Siempre es mejor usar el patrón Mediator para estos problemas)
- Tenemos que contar con toda la información del contexto del problema disponible
- Hay que hacerse preguntas (Reflection) antes de tomar una decisión e indicar porqué escogemos una opción y no otra
- Comprobar si las alternativas a una decisión de diseño son coherentes y tienen relación entre ellas (ej. sugerir diversos patrones de diseño sin relación entre ellos o con el problema)

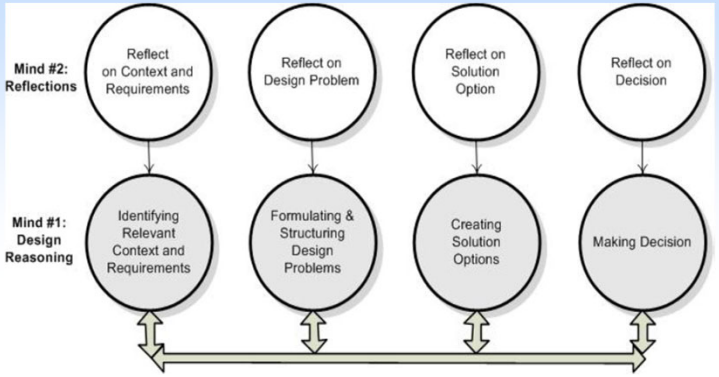
Reflection

Modelo Mind1/Mind2

Es un modelo en el que las personas que tomas las decisiones (Mind1), éstas son desafiadas y puestas en duda por otras personas (Mind2) para obtener decisiones con más calidad



Reflection

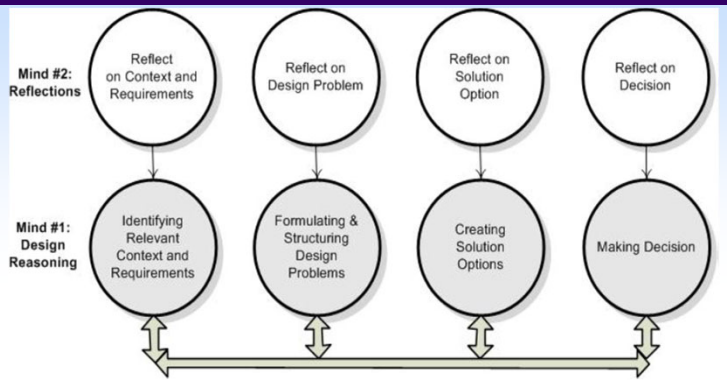


Mind 1

- **Identificar el contexto relevante y requisitos**: Recoger información del contexto de problema, factores que afectan a las decisiones y requisitos funcionales y no funcionales.
- **Formular y estructurar problemas de diseño**: El arquitecto software debe plantear los problemas de diseño principales.
- **Plantear soluciones y alternativas**: Cada solución parcial resuelve problemas de diseño. Cuando se toman decisiones pueden aparecer problemas nuevos.
- **Seleccionar una solución**: Los diseñadores realizan una elección lógica después de analizar y evaluar opciones.

Reflection

Mind 2



- **Reflexionar sobre contextos y requisitos**: Se reflexiona sobre los contextos y requisitos hasta que estos sean completos, relevantes y precisos.
- **Reflexionar sobre problemas de diseño**: Se evalúan si los problemas de diseño están bien articulados y la reflexión debe desafiar a problemas no bien articulados.
- **Reflexionar sobre soluciones de diseño**: Desafía a las soluciones de diseño y sus alternativas (e.g., plantear soluciones sin alternativas, soluciones aceptadas sin requisitos, soluciones que no resuelven ningún requisito).
- **Reflexionar sobre decisiones de diseño**: Se reflexiona sobre la validez de la solución de diseño, los pros y los cons, riesgos, etc.

Conclusiones

- Las decisiones de diseño ayudan a comprender el porqué del proceso de construcción arquitectónica
- Difíciles de administrar cuando el número de decisiones aumenta. Se debe capturar aquellas que sean claves
- Permiten determinar el porqué de decisiones erróneas o sub-óptimas tomadas
- Permiten conocer cuáles fueron las alternativas de diseño
- La reflexión en las decisiones nos permite evaluar su calidad y el porqué de las decisiones tomadas
- **Múltiples barreras en la adopción de AK**
  - Falta de motivación e incentivos
  - La utilidad no se percibe
  - Carencia de herramientas adecuadas
  - Sobrecarga en la captura de decisiones

Lecturas adicionales

- Herramienta ADMentor: https://www.ost.ch/en/research-and-consulting-services/computer-science/ifs-institute-for-software-new/cloud-application-lab/architectural-knowledge-management-akm/admentor-tool
- Blog OZ: https://ozimmer.ch/practices/2020/04/27/ArchitectureDecisionMaking.html
- Formato MADR: https://adr.github.io/madr/
- ADR Manager: https://adr.github.io/adr-manager/#/